UNIT 19 90 min

Machine Learning with scikit-learn

The supervised-learning workflow: features and labels, train/test splits, fitting models, measuring accuracy, and avoiding overfitting.

By the end of this unit you will be able to

- Explain features, labels, and the difference between supervised and unsupervised learning
- Split data into training and test sets, and explain why the split must come first
- Fit a model with `fit`, make predictions with `predict`, and measure it with `score`
- Read a confusion matrix and explain what accuracy does and does not tell you
- Recognize overfitting and use a pipeline so preprocessing does not leak test information

What machine learning is

Machine learning finds patterns in data and uses them to make predictions. Instead of writing the rules yourself, you give the algorithm examples and let it estimate the rules. The quality of a model depends mostly on the data and on how honestly you measure it.

- **Features** are the input measurements, such as petal length or hours studied. Usually a table with one row per example.
- **Label** (or target) is the answer you want to predict, such as the flower species.
- **Supervised learning** uses labeled examples. Classification predicts a category; regression predicts a number.
- **Unsupervised learning** finds structure without labels, such as grouping similar customers. It is covered briefly in the capstone.

Load a dataset

scikit-learn ships with small classic datasets, so you can start without downloading anything. The iris dataset has 150 flowers, four measurements each, and three species.

load_iris.py

```
from sklearn.datasets import load_iris

iris = load_iris()
print(iris.feature_names)
print(iris.target_names)
print(iris.data.shape)
print(iris.data[0])
print(iris.target[0])
```

`iris.data` is the feature table (150 rows, 4 columns). `iris.target` holds the label for each row as a number: 0, 1, or 2. `iris.target_names` says what each number means.

Train and test sets

A model must be tested on examples it has never seen. Hold back part of the data as a **test set** before any training. Train on the rest. If you test on the data you trained on, the score measures memory, not learning.

split.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)
print(len(X_train), len(X_test))
```

> **Note**
> By convention, `X` (capital) holds the features and `y` holds the labels. `random_state=42` makes the split the same every run. The number 42 is arbitrary; any fixed number works.

Fit, predict, score

Every scikit-learn model follows the same pattern. Create it, call `fit(X_train, y_train)` to learn from the training data, call `predict(X_test)` to get predictions, and call `score(X_test, y_test)` to get accuracy. Once you learn this pattern, you can swap models freely.

first_model.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

predictions = model.predict(X_test)
print(predictions[:10])
print("accuracy:", round(model.score(X_test, y_test), 3))
```

K-nearest neighbors predicts the majority label among the *k* closest training examples. It is simple to understand, which makes it a good first model.

Accuracy and the confusion matrix

Accuracy is the fraction of correct predictions. It can hide problems: if 95% of emails are not spam, a model that always says "not spam" scores 95%. A **confusion matrix** shows where each kind of mistake happens. Rows are the true labels, columns are the predictions.

confusion.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)
model = KNeighborsClassifier(n_neighbors=3).fit(X_train, y_train)
pred = model.predict(X_test)

print(accuracy_score(y_test, pred))
print(confusion_matrix(y_test, pred))
```

Tip

Chaining `.fit(...)` works because `fit` returns the model itself. It is a common pattern in scikit-learn.

Overfitting

Overfitting means the model learned the training examples, including their noise, and does poorly on new data. A model with a very large `k` is too simple; a model with `k=1` can memorize. Compare training and test scores to see the gap. A large gap is a warning.

overfit.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)

for k in [1, 3, 5, 15]:
    model = KNeighborsClassifier(n_neighbors=k).fit(X_train, y_train)
    train_acc = model.score(X_train, y_train)
    test_acc = model.score(X_test, y_test)
    print(k, round(train_acc, 2), round(test_acc, 2))
```

Preprocessing with a pipeline

Some models need features on a similar scale. `StandardScaler` rescales each feature to mean 0 and standard deviation 1. The scaler must learn its numbers from the **training** data only. If you scale the whole table first, information from the test set leaks into training, and the score is too optimistic. A `Pipeline` does the steps in the right order for you.

pipeline.py

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42
)

pipe = make_pipeline(StandardScaler(), LogisticRegression(max_iter=500))
pipe.fit(X_train, y_train)
print("accuracy:", round(pipe.score(X_test, y_test), 3))
```

Regression: predicting a number

When the label is a number, use a regressor. `LinearRegression` fits a straight line (or plane) through the data. The score for regression is R^2, where 1.0 is perfect and 0 means no better than predicting the average.

regression.py

```
from sklearn.linear_model import LinearRegression

hours = [[1], [2], [3], [4], [5]]
scores = [52, 58, 65, 71, 76]

line = LinearRegression().fit(hours, scores)
print(round(line.coef_[0], 2))
print(round(line.predict([[6]])[0], 1))
```

Predict the output THINK FIRST

Trust the process: predict what the model will do with each change.

```
from sklearn.datasets import load_iris
print(len(load_iris().target))
```

Answer: `150`

```
import numpy as np
print(np.bincount([0, 0, 1, 2]))
```

Answer: `[2 1 1]`
Counts how often each label value appears.

```
from sklearn.linear_model import LinearRegression
m = LinearRegression().fit([[0], [1], [2]], [0, 2, 4])
print(round(m.predict([[10]])[0]))
```

Answer: `20`
The data follows y = 2x exactly, so the fitted line does too.

Practice

▼ **Exercise 1 — Try other k values**

Using the iris split from this unit (`random_state=42` , `test_size=0.25`), find which value of `k` from 1 to 15 gives the highest test accuracy. Print that `k` and its accuracy.

▼ **Hint**

Loop over `range(1, 16)` , keep track of the best score and its k.

▼ Exercise 2 — Spot the leak

The code below scales all the data before splitting. Explain in one sentence what is wrong with it, then rewrite it so the scaler is fitted on the training set only.

```
X_scaled = StandardScaler().fit_transform(iris.data)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, iris.target, r
```

Discussion

1. A spam filter is 99% accurate. Why might that still be a bad filter? What would you check next?
2. Why must the test set stay untouched until the very end of a project? What happens to your honest score if you tune the model using it?
3. Which features in the iris data would a person use to tell species apart? Does the model use them the same way?

Key terms

Feature

An input variable used to make a prediction, usually a column in the feature table.

Label / target

The value a supervised model is trained to predict.

Supervised learning

Learning from examples that include the correct label.

Unsupervised learning

Finding structure in data that has no labels, such as clusters.

Train/test split

Dividing data into a set used for fitting and a held-back set used only for evaluation.

Accuracy

The fraction of predictions that match the true labels.

Confusion matrix

A table of true labels against predicted labels, showing which mistakes the model makes.

Overfitting

When a model fits the training data, including its noise, so well that it performs worse on new data.

Data leakage

When information from the test set influences training, making the measured score too optimistic.

Pipeline

A scikit-learn object that chains preprocessing steps and a model so they are fitted and applied in order.

R^2

A regression score where 1.0 is a perfect fit and 0 means no better than predicting the average.

See all terms in the glossary →